Hardware Information Commands

Hardware information commands help provide insight into various hardware devices on the system. Use these commands to check **hardware** information and see hardware device statuses.

Command	Description
`lscpu`	See **CPU** information.
`lsblk`	See information about **block devices**.
`lspci -tv`	**Show PCI devices** (graphics card, network card, etc.) in a tree-like diagram.
`lsusb -tv`	Display USB devices in a tree-like diagram.
`lshw`	List hardware configuration information.
`cat /proc/cpuinfo`	Show detailed CPU information.
`cat /proc/meminfo`	View detailed system memory information.
`cat /proc/mounts`	See mounted **file systems**.
`free -h`	Display **free and used memory**.
`sudo dmidecode`	Show hardware information from the **BIOS**.
`hdparm -i /dev/[device_name]`	Display disk data information.
`hdparm -tT /dev/[device_name]`	Conduct a read speed test on the device/disk.
`badblocks -s /dev/[device_name]`	Test for unreadable blocks on the device/disk.
`fsck /dev/[device_name]`	**Run a disk check** on an unmounted disk or partition.

Searching Commands

Linux offers various commands to search for files, directories, and text. Use these commands to search for files and directories on the system and filter the search using various patterns.

Command	Description
`find [path] -name [search_pattern]`	**Find files and directories** that match the specified pattern in a specified location.
`find [path] -size [+100M]`	See files and directories larger than a specified size in a directory.
`grep [search_pattern] [file_name]`	**Search for a specific pattern** in a **file** with **grep**.
`grep -r [search_pattern] [directory_name]`	Recursively search for a pattern in a directory.
`locate [name]`	**Locate all files and directories** related to a particular name.
`which [command]`	**Search the command path** in the `$PATH` environment variable

`whereis [command]`	Use the **whereis command** to find the source, binary, and manual page for a command.
`awk '[search_pattern] {print $0}' [file_name]`	**Print all lines matching a pattern** in a file. See also the **gawk command**, the **GNU** version of `awk`.
`sed 's/[old_text]/[new_text]/' [file_name]`	**Find and replace text** in a specified file.

File Commands

File commands help with file and directory management on the system. Create, delete, move, and modify files and directories from the terminal using the commands in the following table.

Command	Description
`mkdir [directory_name]`	**Create a new directory**.
`rm [file_name]`	**Remove a file**.
`rm -r [directory_name]`	Remove a directory recursively.
`rm -rf [directory_name]`	Recursively remove a directory without requiring confirmation.
`cp [source_file] [destination_file]`	**Copy the contents of one file** to another file using the **cp command**.
`cp -r [source_directory] [destination_directory]`	Recursively copy a directory to a second directory.
`mv [source_file] [destination_file]`	**Move or rename files** or directories.
`ln -s [path]/[file_name] [link_name]`	**Create a symbolic link** to a file.
`touch [file_name]`	Create a new file using **touch**.
`cat [file_name]`	**Show the contents of a file**.
`cat [source_file] >> [destination_file]`	Append file contents to another file.
`head [file_name]`	Show the first ten lines of a file.
`tail [file_name]`	Show the last ten lines of a file with the **tail command**.
`more [file_name]`	Display contents of a file page by page.
`less [file_name]`	Show the contents of a file with navigation using the **less command**.
`nano [file_name]`	Open or create a file using the **nano text editor**.
`vi [file_name]` `vim [file_name]`	Open or create a file using the **Vi/Vim text editor**.
`gpg -c [file_name]`	Encrypt a file.
`gpg [file_name].gpg`	Decrypt an encrypted *.gpg* file.

`wc -w [file_name]`	Show the number of words, lines, and bytes in a file using **wc**.
`ls | xargs wc`	List the number of lines/words/characters in each file in a directory with the **xargs command**.
`cut -d [delimiter] [file_name]`	**Cut a section of a file** and print the result to standard output.
`[data] | cut -d [delimiter]`	Cut a section of piped data and print the result to standard output.
`shred -u [file_name]`	**Overwrite a file** to prevent its recovery, then delete it.
`diff [first_file] [second_file]`	**Compare two files** and display differences.
`source [file_name]`	**Read and execute the file content** in the current shell.
`[command] | tee [file_name] >/dev/null`	**Store the command output in a file** and skip the terminal output.

Directory Navigation Commands

Directory navigation commands provide shortcuts to navigate to the desired location quickly. Below are several crucial shortcuts to remember when navigating directories in Linux through the terminal.

Directory Navigation Commands	Description
`ls`	**List files and directories** in the current directory.
`ls -a`	List all files and directories in the current directory (**shows hidden files**).
`ls -l`	List files and directories in long format.
`pwd`	**Show the directory** you are currently working in.
`cd` `cd ~`	**Change directory** to `$HOME`.
`cd ..`	Move up one directory level.
`cd -`	Change to the previous directory.
`cd [directory_path]`	Change location to a specified directory.
`dirs`	Show current directory stack.

File Compression Commands

Archive and compress files to save space on your system and organize your data. There are various compression formats available on Linux, and the table below shows the most commonly used file compression commands.

Command	Description
`tar cf [archive.tar] [file/directory]`	Archive an existing file or directory.
`tar xf [archive.tar]`	**Extract an archived file**.
`tar czf [archive.tar.gz]`	Create a *.gz* compressed tar archive.
`gzip [file_name]` `gunzip [file_name.gz]`	**Compress** or decompress *.gz* files.
`bzip2 [file_name]` `bunzip2 [file_name.bz2]`	Compress or **decompress .bz2 files**.

File Transfer Commands

Depending on the transfer method and protocol, Linux has various file transfer commands. Use these commands to quickly transfer files between two remote machines or to download files from web servers.

Command	Description
`scp [source_file] [user]@[remote_host]:[destination_path]`	Copy a file to a server directory securely using the **Linux scp command**.
`rsync -a [source_directory] [user]@[remote_host]:[destination_directory]`	Synchronize the contents of a directory with a backup directory using the **rsync command**.
`wget [link]`	Download files from **FTP** or web servers via the **wget command**.
`curl -O [link]`	Transfer data to or from a server with various protocols using the **curl command**.
`ftp [remote_host]`	Transfer files between local and remote systems interactively using FTP.
`sftp [user]@[remote_host]`	Securely transfer between local and remote hosts using **SFTP**.

User and Group Commands

Linux is a multiuser system, and there are various commands to add, modify, remove, and manage users and groups. The table below outlines the critical user and group commands.

Command	Description
`id`	See details about the active users.
`last`	Show the last system logins.
`who`	Display who is currently logged into the system with the **who command**.
`w`	Show which users are logged in and their activity with the **w command**.
`finger [user_name]`	Show user information.
`sudo useradd [user_name]`	Create a new user account.
`sudo adduser [user_name]`	Create a new user account through the **adduser command** interface.
`sudo userdel [user_name]`	Delete a user account.
`sudo usermod -aG [group_name] [user_name]`	**Modify user information** (**add a user to a group**).
`passwd` `sudo passwd [user_name]`	Change the current user's password or another user's password with the **passwd command**.
`sudo groupadd [group_name]`	Add a new group.
`sudo groupdel [group_name]`	Delete a group.
`sudo groupmod -n [new_name] [old_name]`	Modify a user group (change group name).
`sudo [command]`	Temporarily elevate user privileges to superuser or root using the **sudo command**.
`su - [user_name]`	**Switch the user account** or become a superuser.
`chgrp [group_name] [file/directory]`	**Change file or directory group**.

Package Installation Commands

Specific package manager commands vary between different **Linux distributions**. Choose the commands that match your specific distribution.

Debian and Ubuntu-Based Distributions

Debian and Ubuntu-based distributions use the `apt-get`, `apt`, and `dpkg` commands to manage packages. Below are crucial commands necessary to install packages on these systems.

Command	Description
`sudo apt-get install [package_name]`	Install an APT package using the **apt-get package utility**.
`sudo apt install [package_name]`	Install an APT package using a **newer APT package manager**.
`apt search [keyword]`	Search for a package in the APT repositories.
`apt list`	List packages installed with APT.
`apt show [package_name]`	Show information about a package.
`sudo dpkg -i [package_name.deb]`	Install a *.deb* package with the Debian package manager (**dpkg command**).
`sudo dpkg -l`	List packages installed with dpkg.

Red Hat-Based Distributions (CentOS, Fedora)

Red Hat-based distributions, including CentOS and Fedora, use the `yum`, `dnf`, and `rpm` commands for package installation. The table below provides the syntax for each option.

Command	Description
`sudo yum install [package_name]`	Install a package using the YUM package manager.
`yum search [keyword]`	Find a package in the YUM repositories based on the provided keyword.
`yum list installed`	**List all packages installed with YUM**.
`yum info [package_name]`	Show package information for a package.
`sudo dnf install [package_name]`	Install a package using the DNF package manager.
`sudo rpm -i [package_name.rpm]`	Install a ***.rpm* package** from a local file.

Universal Linux Packages

Universal Linux packages are distribution-independent package collections. The two most popular options that come preinstalled on many systems are `snap` and `flatpak` commands. Alternatively, most **programs** offer the option to install from source. The table below shows the syntax for each listed option.

Command	Description

`tar zxvf [file_name.tar.gz]`
`cd [extracted_directory]`
`./configure make`
`make install`
	Install software from **source code**.

`sudo snap install [package_name]`	Install a **Snap** package.
`sudo snap find [keyword]`	Search for a package in the Snap store.
`sudo snap list`	List installed Snap packages.
`flatpak install [package_name]`	Install a Flatpak package.
`flatpak search [keyword]`	Search for a Flatpak application in repositories.

Process Related Commands

Every application and command in Linux creates a process. The table below shows various process-related commands to help manage Linux processes.

Command	Description
`ps`	**List active processes**.
`pstree`	Show processes in a tree-like diagram.
`pmap`	Display a memory usage map of processes.
`top`	See **all running processes**.
`htop`	Interactive and colorful process viewer.
`kill [process_id]`	**Terminate a Linux process** under a given ID.
`pkill [process_name]`	Terminate a process under a specific name.
`killall [label]`	Terminate all processes with a given label.
`prgrep [keyword]`	List processes based on the provided keyword.

`pidof [process_name]`	Show the PID of a process.
`bg`	List and resume stopped jobs in the background.
`fg`	Bring the most recently suspended job to the foreground.
`fg [job]`	Bring a particular job to the foreground.
`lsof`	List files opened by running processes with **lsof command**.
`trap "[commands]" [signal]`	**Catch a system error signal** in a shell script. Executes provided commands when the signal is caught.
`wait`	**Pause the terminal or a Bash script** until a running process is completed.
`nohup [command] &`	**Run a Linux process** in the background.

System Management and Information Commands

Use the terminal to manage the system directly. The commands show how to view basic system information, change options, and reboot or **restart the system**.

Command	Description
`uname -r`	Show system information via **uname command**.
`uname -a`	See **kernel release information**.
`uptime`	**Display system uptime**, including the load average.
`hostname`	View system hostname.
`hostname -i`	Show the **IP address** of the system.
`last reboot`	List system reboot history.
`date`	See **current time and date**.
`timedatectl`	Query and change the **system clock**.

`w`	List logged-in users.
`whoami`	See which user you are using.
`finger [user_name]`	Show information about a particular user.
`ulimit [flags] [limit]`	View or limit system resource amounts.
`shutdown [hh:mm]`	Schedule a system shutdown.
`shutdown now`	Shut down the system immediately.
`modprobe [module_name]`	Add a new kernel module.
`dmesg`	Show bootup messages.

Disk Usage Commands

Disk usage commands provide insight into disk space status. You can use the `df` and du commands to check disk space in Linux. Common disk usage commands are outlined in the table below.

Command	Description
`df -h`	Check free and used space on mounted systems.
`df -i`	Show free inodes on mounted file systems.
`fdisk -l`	Display disk partitions, sizes, and types with the command.
`du -ah`	See disk usage for all files and directories.
`du -sh`	Show disk usage of the current directory.
`mount`	Show currently mounted file systems.
`findmnt`	Display target mount point for all file systems.
`mount [device_path] [mount_point]`	Mount a device.
`sftp [user_name]@[remote_host]`	Interactive file transfer over encrypted SSH session using SFTP protocol.
`telnet [host]`	Connect to the host via Telnet default port 23.

SSH Login Commands

SSH commands enable connecting to a remote host using the SSH protocol. Other commands use this protocol for copying and transferring files between two systems.

See the table below for common SSH commands. For a detailed explanation of SSH Linux Commands, refer to our **SSH Commands in Linux** tutorial.

Command	Description
`ssh [user_name]@[remote_host]`	**Connect to a remote host** as a user via SSH.
`ssh [host]`	Securely connect to a host via **SSH default port 22**.
`ssh -p [port] [user_name]@[remote_host]`	Connect to the host using a particular port.
`ssh-keygen`	**Generate SSH key pairs**.
`sudo service sshd start`	Start SSH server **daemon**.
`scp [file_name] [user_name]@[remote_host]:[remote_path]`	**Securely copy files** between local and remote systems via SSH.
`sftp [user_name]@[remote_host]`	Interactive file transfer over encrypted SSH session using **SFTP** protocol.
`telnet [host]`	Connect to the host via **Telnet** default port 23.

File Permission Commands

File permission commands control user permissions over files and directories. Use these commands to change and manage the owner, group, and user permissions on the system.

Command	Description
`chmod 777 [file_name]`	Assign read, write, and execute **file permission** to everyone (`rwxrwxrwx`).
`chmod 755 [file_name]`	Give read, write, and execute permission to owner, and read and execute permission to group and others (`rwxr-xr-x`).
`chmod 766 [file_name]`	Assign full permission to the owner, and read and write permission to the group and others (`rwxrw-rw-`).
`chown [user_name] [file_name]`	Change the ownership of a file with **chown command**.
`chown [user_name]:[group_name] [file_name]`	Change the owner and group ownership of a file.

Network Commands

The table below shows a brief list of common Linux network commands. For a more comprehensive list, check out our **Linux Network Commands Cheat Sheet**.

Description	Command
`ip addr show`	**List IP addresses** and network interfaces.
`ip address add [IP_address]`	Assign an IP address to interface **eth0**.
`ifconfig`	Display IP addresses of all network interfaces.
`ping [remote_host]`	**Ping** remote host.
`netstat -pnltu`	See active (listening) ports with the **netstat command**.
`netstat -tuln`	Show **TCP** and **UDP** ports and their programs.
`whois [domain_name]`	Display more information about a **domain**.
`dig [domain_name]`	Show **DNS** information about a domain using the **dig command**.
`dig -x [domain_name]`	Do a reverse DNS lookup on the domain.
`dig -x [IP_address]`	Do a **reverse DNS lookup** of an IP address.
`host [domain_name]`	Perform an IP lookup for a domain.
`hostname -I`	Show the local IP address.
`nslookup [domain_name]`	Receive **information about an internet domain**.

Shell Command Management

Several Linux commands enable managing command execution flow. Schedule commands or create shorter names for lengthy commands.

Command	Description
`alias [alias-name]='[command]'`	**Create an alias** for a command.
`watch -n [interval-in-seconds] [command]`	**Set a custom interval** to run a user-defined command.
`sleep [time-interval] && [command]`	**Postpone the execution** of a command.
`at [hh:mm]`	Create a job to be executed at a certain time (**Ctrl+D** to exit prompt after you type in the command).
`man [command]`	**Display a built-in manual** for a command.
`history`	Print the **command history** used in the terminal.

Variable Commands

Shell variables are a convenient way to store critical data in a reusable format. Below are commands to create, view, and remove variables in the shell.

Command	Description
`let "[variable_name]=[value]"`	Assign an integer value to a variable.
`export [variable_name]`	Export a Bash variable.
`declare [variable-name]= "[value]"`	Declare a Bash variable.
`set`	List the names of all the shell variables and functions.
`unset [variable_name]`	Remove an environment variable.
`echo $[variable-name]`	Display the value of a variable.

Linux Shell Keyboard Shortcuts

The Linux shell contains several keyboard shortcuts that are important to know. The commands help control the terminal behavior in specific situations. The table below outlines the most important shortcuts.

Shortcut	Description
Ctrl + **C**	Kill process running in the terminal.
Ctrl + **Z**	Stop the current process. The process can be resumed in the foreground with **fg** or in the background with **bg**.
Ctrl + **W**	Cut one word before the cursor and add it to the clipboard.
Ctrl + **U**	Cut part of the line before the cursor and add it to the clipboard.
Ctrl + **K**	Cut part of the line after the cursor and add it to the clipboard.
Ctrl + **Y**	Paste from clipboard.
Ctrl + **R**	Recall the last command that matches the provided characters.
Ctrl + **O**	Run the previously recalled command.
Ctrl + **G**	Exit command history without running a command.
`clear`	Clear the terminal screen.
`!!`	Run the last command again.
`exit`	Log out of the current session.